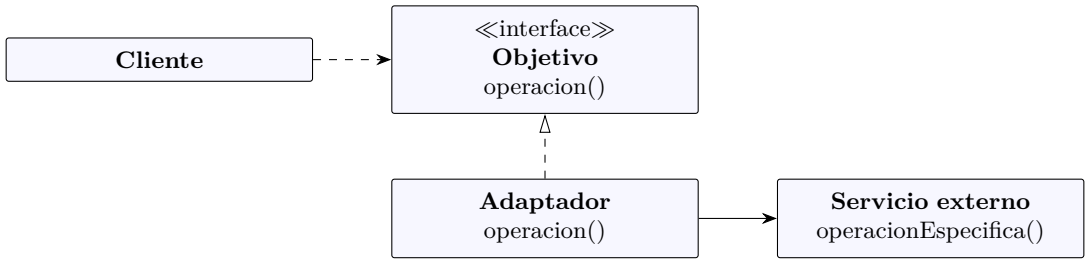
1. Adapter

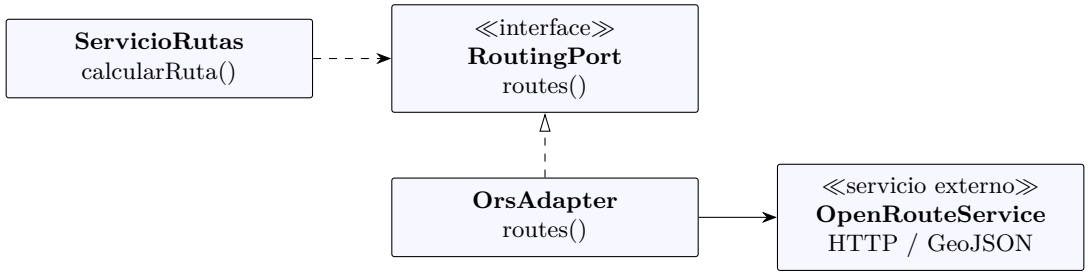
Para calcular una ruta necesitamos enviar los datos a OpenRouteService y leer su respuesta en GeoJSON. Esos formatos son propios de la API, así que usamos un adaptador para convertirlos a los tipos con los que trabaja Vía.

El servicio usa la interfaz `RoutingPort`, que define la operación de consultar rutas. `OrsAdapter` cumple esa interfaz y se encarga de la comunicación con ORS. Ya lo utilizamos en el spike y lo hemos pasado a `ServicioRutas` en las pruebas de la HU-14. También tiene las operaciones de geocodificación necesarias para buscar lugares.

UML genérico



UML aplicado a Vía



Ventajas. Los detalles de ORS quedan dentro del adaptador. Si cambiamos de proveedor, podemos escribir otro adaptador que ofrezca la misma interfaz y mantener las llamadas de `ServicioRutas`.

Inconvenientes. Tenemos que mantener las conversiones de datos y errores. Además, la interfaz debe recoger lo que necesita Vía para poder usarla con otros proveedores.

Alternativa. Podríamos hacer las llamadas a ORS dentro del servicio de rutas. Eso ahorraría una clase, pero obligaría al servicio a conocer las peticiones HTTP y el formato GeoJSON. Hemos elegido Adapter para concentrar esa parte en un sitio.

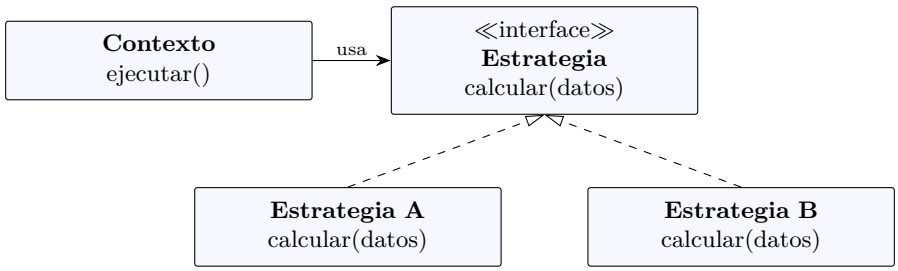
Código. `OrsAdapter` está en `src/adapters/providers.ts` y `RoutingPort` en `src/domain/types.ts`. En estos diagramas mostramos las operaciones de rutas.

2. Strategy

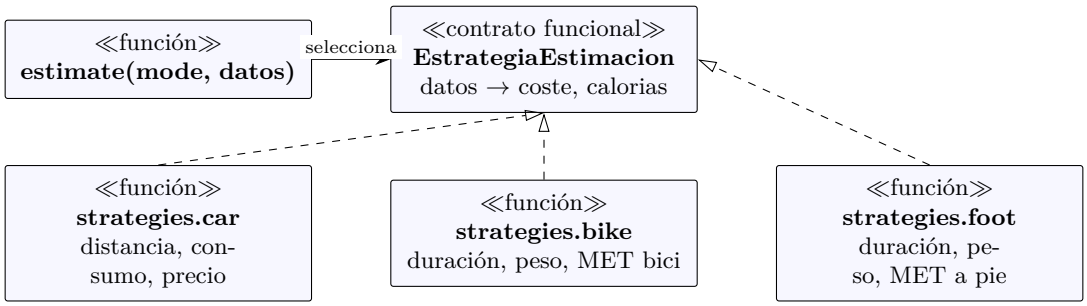
La estimación del viaje depende del transporte. Para un coche calculamos el gasto con la distancia, el consumo del vehículo y el precio del combustible o de la electricidad. Al ir en bicicleta o a pie, calculamos calorías a partir de la duración, el peso y el MET de la actividad.

Usamos Strategy para escoger el cálculo según el modo de transporte. En el spike, `estimate` selecciona una de las funciones del objeto `strategies`. Todas devuelven coste y calorías. Queremos mantener esta organización al desarrollar las HU-15, HU-16 y HU-17.

UML genérico



UML aplicado a Vía



Ventajas. Podemos leer y probar las fórmulas de cada transporte por separado. El planificador llama a la misma operación y recibe el resultado del cálculo elegido.

Inconvenientes. Las estrategias deben aceptar los datos necesarios y devolver un resultado común. Conviene mantener juntas las fórmulas que apenas cambian para evitar repetir cálculos.

Alternativa. Un condicional en el planificador permitiría elegir la fórmula, pero haría crecer ese método al añadir cálculos. En nuestro caso, la separación se hace por transporte. Combustible y electricidad usan la misma fórmula, distancia/100 × consumo × precio, con unidades distintas.

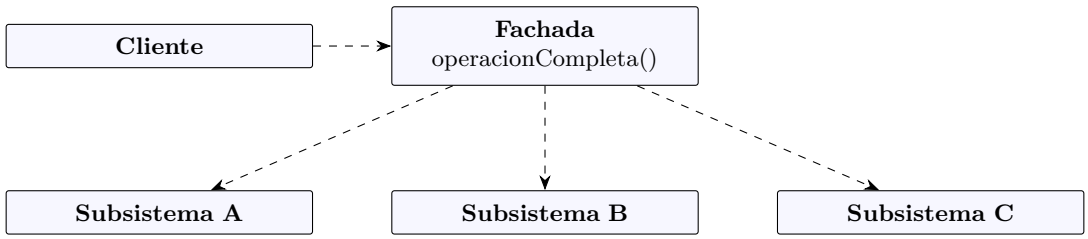
Código. Los cálculos están en `src/domain/cost.ts`. En el UML llamamos `EstrategiaEstimacion` a la firma común de esas funciones.

3. Facade

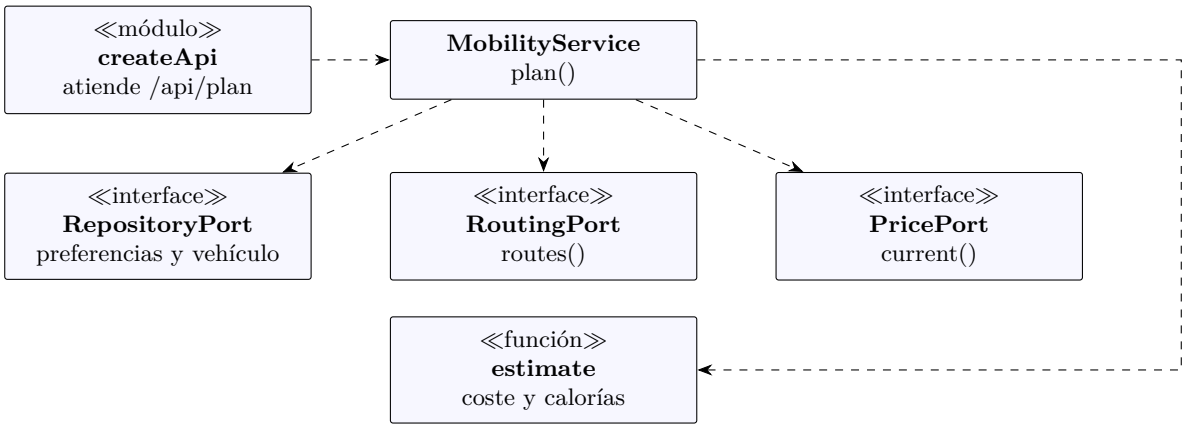
Cuando el usuario pide un viaje, hay que comprobar el origen y el destino, recuperar sus preferencias y su vehículo, consultar rutas y precios, calcular el gasto y ordenar las alternativas. Necesitamos coordinar todos esos pasos antes de devolver el resultado.

La fachada ofrece una operación de planificación que reúne ese trabajo. En el spike, el manejador HTTP llama a `MobilityService.plan` y este coordina el repositorio, el proveedor de rutas, los precios y los cálculos. Este diseño cubre el cálculo de ruta de la HU-14 y las estimaciones y criterios de las HU-15 a HU-18.

UML genérico



UML aplicado a Vía



Ventajas. Desde el manejador HTTP basta con llamar a `plan`. La fachada establece el orden de los pasos y cada servicio realiza su consulta o cálculo.

Inconvenientes. La fachada puede acabar haciendo demasiado. En el spike, `MobilityService` ya reúne muchas operaciones. Al desarrollar el proyecto tendremos que mantener los cálculos y las consultas en sus componentes para que la fachada se centre en coordinarlos.

Alternativa. Podríamos coordinar estos pasos desde el manejador HTTP, aunque eso haría más difícil reutilizar la planificación desde otro punto. Con la fachada, esa operación queda disponible para otros clientes.

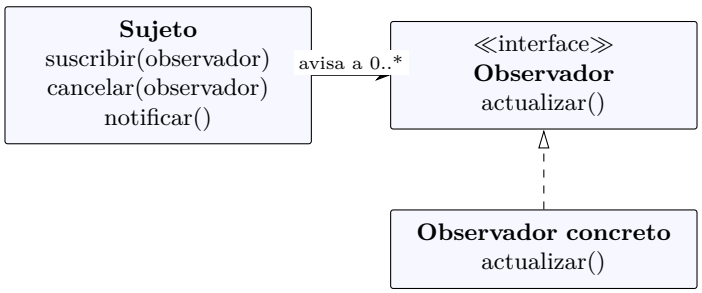
Código. El método `plan` está en `src/services.ts` y se llama desde `src/http.ts`. El diagrama recoge las dependencias de esa operación.

4. Observer

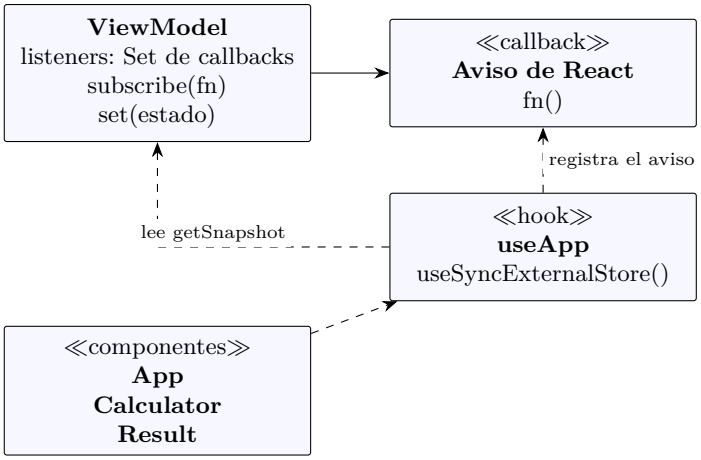
Las vistas muestran datos del mismo usuario y de la misma planificación. Por ejemplo, cuando llega una ruta calculada, la vista de resultados debe actualizarse. Para ello, el modelo guarda las funciones de aviso de las vistas suscritas y las llama cuando cambia el estado.

En el spike, los componentes `App`, `Calculator` y `Result` se suscriben mediante `useApp`. Este hook conecta React con `ViewModel.subscribe`. Cuando `set` cambia el estado, el modelo ejecuta los avisos y React actualiza los componentes. `subscribe` también devuelve una función para cancelar la suscripción.

UML genérico



UML aplicado a Vía



Ventajas. El modelo puede avisar de un cambio a todas las vistas interesadas sin conocer qué componente representa cada una. Esto permite que varias partes de la pantalla sigan el mismo estado.

Inconvenientes. Hay que cancelar las suscripciones cuando se desmontan los componentes. Además, si todos escuchan el mismo estado, un cambio pequeño puede hacer que se actualicen más vistas de las necesarias.

Alternativa. Podríamos guardar el estado en un componente padre y pasarlo a sus hijos mediante propiedades. Esa solución serviría para una interfaz pequeña. Queremos aprovechar el modelo observable del spike porque ya permite compartir los cambios con las tres vistas.

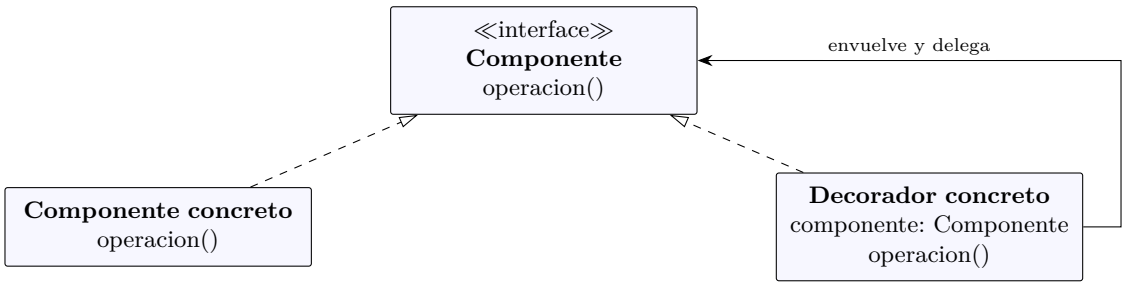
Código. `ViewModel` y `useApp` están en `web/model.ts`. Los componentes que los usan están en `web/main.tsx`. En el UML hemos señalado qué elementos son funciones.

5. Decorator

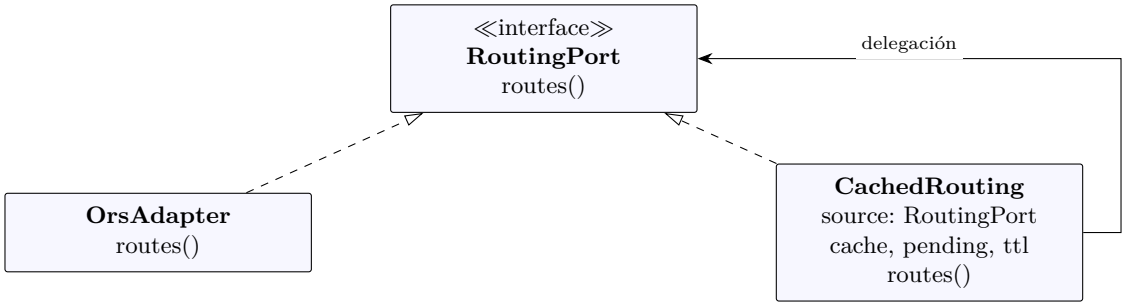
Si el usuario vuelve a pedir la misma ruta, podemos aprovechar el resultado reciente para ahorrar otra llamada a la API. También nos interesa evitar que dos peticiones iguales que llegan a la vez provoquen dos consultas al proveedor.

Para añadir ese comportamiento usamos `CachedRouting`, que envuelve al proveedor y ofrece el mismo método `routes`. En el spike conserva hasta cien resultados durante tres minutos y agrupa las peticiones simultáneas iguales. Queremos mantener esta caché en el proyecto para reducir las consultas repetidas.

UML genérico



UML aplicado a Vía



El servidor lo configura con `new CachedRouting(ors)`. Al consultar una ruta, el decorador busca las coordenadas de origen y destino y el modo de transporte en su caché. Si el resultado tiene menos de tres minutos, devuelve una copia. Si falta o ha caducado, consulta al proveedor. Los errores se descartan para que una nueva petición pueda volver a intentarlo.

Ventajas. Ahorramos llamadas a la API y reducimos la espera en consultas repetidas. Los servicios pueden seguir usando `routes` con la misma interfaz.

Inconvenientes. La caché ocupa memoria y puede conservar una ruta que haya cambiado en el proveedor. Por eso limitamos el número de resultados y su duración.

Alternativa. Podríamos guardar la caché dentro de `OrsAdapter`. Con un decorador podemos usar la misma caché alrededor de otro proveedor que cumpla `RoutingPort`, y decidir al crear el servicio si queremos activarla.

Código. `CachedRouting` está en `src/adapters/providers.ts` y se configura en `src/server.ts`. Sus pruebas están en `tests/adapters.test.ts`.

Este apartado recoge los cinco patrones que prevemos usar, a partir de las historias y del spike. El SUT de la IT1 sigue en fase roja. La elección del patrón de interfaz queda pendiente de documentar en su apartado.